ChatGPT

Production Backend Microservices Plan (HRMS + Task Module)

Executive Summary

We propose a **Spring Boot microservices** architecture using **MySQL**, **Liquibase**, **Swagger/OpenAPI**, and **Keycloak** for an HRMS with Task module. Core domains (Auth, Users, Employees, Tasks, Payroll, Notifications) each become separate services (plus shared libraries). Keycloak provides authentication/authorization (we'll include a Keycloak provider library). Each service uses Liquibase changelogs to set up its MySQL schema. APIs will be documented and generated via Swagger. Common DTOs reside in a shared library for uniform API responses.

Below we outline: required features (HR, Payroll, Task), recommended service boundaries (names and responsibilities), API designs with Swagger, database schema (DDL via Liquibase), integration of Keycloak, and the steps for AI-backed code generation. An explicit AI prompt and developer README instructions are provided at the end to generate the full backend.

1. Feature List

HR Module: Multi-tenant support; employee profiles; departments; attendance (punch/shift); leave requests/approvals; organizational config.

User/Identity Module: User accounts, roles/permissions (mapped to Keycloak roles). Tenant settings.

Task Module: Create/assign tasks, subtasks, templates; task priorities and deadlines; time logs per task; manager approvals; comments and attachments; audit trail; notifications. Link tasks to payroll: e.g. approved hours become billable or bonuses.

Payroll Module: Salary structures, tax computations (TDS, PF/ESI, professional tax per India rules), payroll runs (monthly/off-cycle), payslip generation, export reports. Use Indian payroll compliance logic (as in PDF spec).

Notifications: Email/SMS/WhatsApp integration; configurable templates; event-triggered alerts (task due, payroll complete).

Security: OAuth2/JWT with Keycloak; RBAC across services. Field-level encryption for sensitive data.

Logging/Audit: Centralized logs and audit tables capturing all entity changes for compliance.

2. Microservice Architecture

We recommend **6 microservices** plus supporting components:

- **auth-service (Keycloak Provider):** Integrates with Keycloak for login/registration. Contains Keycloak client library (e.g. `com.upcraft.keycloak`) for admin ops (user/realm mgmt).
- **user-service:** Manages application users and tenants. Syncs with Keycloak users.
- **employee-service:** CRUD for employees, departments, attendance, leave.
- **task-service:** Manages tasks, subtasks, time logs, approvals, attachments.
- **payroll-service:** Manages salary config, payroll runs, payslips, tax rules.
- **notification-service:** Sends out emails/SMS/WhatsApp based on events (Task created, payroll run completed).

Each service has its own database schema (in the same MySQL instance, or separate DBs with schema-per-tenant or shared DB with tenant_id). A **gateway/API-service** could also be added for routing (optional).

Shared Libraries:
- `common-dto`: Shared DTO classes for API responses/requests.
- `common-model`: (optional) Shared entities or interfaces.
- `keycloak-provider`: Jar providing Keycloak integration utilities and API wrapper.

Service Responsibilities:
- **Auth Service:** Secures other services; obtains tokens from Keycloak; provides endpoints for token refresh.
- **User Service:** Tenant and user profile management (roles, preferences). Uses Keycloak provider to create Keycloak users.
- **Employee Service:** Employee profiles, org hierarchy, attendance, leave. Publishes events (e.g. `EmployeeCreated`).
- **Task Service:** Task assignment and tracking. Consumes employee info (assignees). Produces events for payroll calculation (e.g. `TaskApproved`).
- **Payroll Service:** Performs payroll runs, listens for task events to add billable hours, computes taxes.
- **Notification Service:** Subscribed to events (`TaskCreated`, `PayrollCompleted`) and uses templates to send messages.

Deployment: Services containerized (Docker) with Kubernetes. Each has its own `liquibase` changelog for schema. Swagger UI on each service (via Springdoc). CI/CD pipeline builds, tests, and deploys each service independently.

3. API Design & DTOs

All services expose RESTful JSON APIs, documented with Swagger (OpenAPI). Each API uses shared DTOs where possible.

Common DTOs: A shared library contains classes like `TenantDTO`, `UserDTO`, `EmployeeDTO`, `TaskDTO`, `SubtaskDTO`, `TimeLogDTO`, `PayslipDTO`, with annotations for Swagger. Reuse these across services.

Auth Service APIs:
- `POST /auth/login` → Keycloak login (returns JWT).
- `GET /auth/refresh` → Refresh token.

User Service APIs: (prefixed `/users`)
- `GET /users` (list all users of tenant).
- `POST /users` (create user; body=UserDTO).
- `PUT /users/{id}`, `DELETE /users/{id}`.
- **Swagger:** all models and paths auto-generated.

Employee Service APIs:
- `GET /employees`, `POST /employees`, `GET/PUT/DELETE /employees/{id}` (uses EmployeeDTO).
- `POST /employees/{id}/attendance` (record punch).
- `POST /leave-requests`, `GET /leave-requests/{id}` etc.

Task Service APIs:
- `GET /tasks`, `POST /tasks`, `GET/PUT/DELETE /tasks/{id}` (TaskDTO with fields: id, title, desc, status, priority, dueDate, assigneeId).
- `POST /tasks/{id}/subtasks`, `GET /tasks/{id}/subtasks`.
- `POST /tasks/{id}/timelogs` (TimeLogDTO with userId, date, hours).
- `POST /tasks/{id}/approve` (approve task).
- Use pagination (`page`, `size`), sorting, and filtering (e.g. by assignee) on list endpoints.

Payroll Service APIs:
- `GET /salary-structures`, `POST /salary-structures`.
- `POST /payroll-runs` (executes payroll for period).
- `GET /payroll-runs/{id}` → returns list of PayslipDTO.

Notification Service APIs: (mostly internal)
- `POST /notifications/email` (send email).
- `POST /notifications/sms`, `/whatsapp`.

Swagger Generation: Use Springdoc (annotations like `@Operation`). A `swagger.yaml` is generated at runtime.

Example DTO (TaskDTO):

```
public class TaskDTO {
    public UUID id;
    public String title;
    public String description;
    public String status; // Pending, InProgress, Completed
    public String priority; // Low, Normal, High
    public UUID assigneeId;
    public LocalDate dueDate;
```

```
    // getters/setters
}
```

4. Database Schema (Liquibase & MySQL)

Each service maintains its own database (or schema). All use Liquibase for migrations.

Common Tenant/User Tables (User Service):

```
<changeSet id="1">
  <createTable tableName="tenant">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="name" type="VARCHAR(100)" />
  </createTable>
</changeSet>
<changeSet id="2">
  <createTable tableName="user">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="tenant_id" type="VARCHAR(36)"/>
    <column name="username" type="VARCHAR(50)" unique="true"/>
    <column name="email" type="VARCHAR(100)" />
    <column name="role" type="VARCHAR(50)" />
    <column name="created_at" type="DATETIME"
defaultValueComputed="CURRENT_TIMESTAMP"/>
  </createTable>
  <addForeignKeyConstraint baseTable="user" baseColumn="tenant_id"
                           referencedTable="tenant" referencedColumn="id"/>
</changeSet>
```

Employee Service Schema:
Liquibase changes for `employee`, `attendance`, `leave_request`. Example:

```
<changeSet id="10">
  <createTable tableName="employee">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="tenant_id" type="VARCHAR(36)"/>
    <column name="user_id" type="VARCHAR(36)"/>
    <column name="department" type="VARCHAR(50)"/>
    <column name="salary" type="BIGINT"/>
    <column name="created_at" type="DATETIME"
defaultValueComputed="CURRENT_TIMESTAMP"/>
  </createTable>
  <addForeignKeyConstraint baseTable="employee" baseColumn="tenant_id"
```

```
                            referencedTable="tenant" referencedColumn="id"/>
</changeSet>
```

(Define similarly for `attendance`, `leave_request`.)

Task Service Schema:
Liquibase changes for `task`, `subtask`, `timelog`, `attachment`:

```
<changeSet id="20">
  <createTable tableName="task">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="tenant_id" type="VARCHAR(36)"/>
    <column name="title" type="VARCHAR(100)"/>
    <column name="description" type="TEXT"/>
    <column name="status" type="VARCHAR(20)"/>
    <column name="priority" type="VARCHAR(10)"/>
    <column name="assignee_id" type="VARCHAR(36)"/>
    <column name="created_by" type="VARCHAR(36)"/>
    <column name="due_date" type="DATE"/>
    <column name="approved_by" type="VARCHAR(36)"/>
    <column name="bonus_amount" type="BIGINT" defaultValueNumeric="0"/>
    <column name="completed_at" type="DATETIME"/>
    <column name="created_at" type="DATETIME"
defaultValueComputed="CURRENT_TIMESTAMP"/>
  </createTable>
  <addForeignKeyConstraint baseTable="task" baseColumn="tenant_id"
                           referencedTable="tenant" referencedColumn="id"/>
</changeSet>
<changeSet id="21">
  <createTable tableName="subtask">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="task_id" type="VARCHAR(36)"/>
    <column name="title" type="VARCHAR(100)"/>
    <column name="status" type="VARCHAR(20)"/>
    <column name="assignee_id" type="VARCHAR(36)"/>
    <column name="due_date" type="DATE"/>
    <column name="created_at" type="DATETIME"
defaultValueComputed="CURRENT_TIMESTAMP"/>
  </createTable>
  <addForeignKeyConstraint baseTable="subtask" baseColumn="task_id"
                           referencedTable="task" referencedColumn="id"/>
</changeSet>
<changeSet id="22">
  <createTable tableName="timelog">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="task_id" type="VARCHAR(36)"/>
```

```
    <column name="user_id" type="VARCHAR(36)"/>
    <column name="log_date" type="DATE"/>
    <column name="hours" type="DECIMAL(5,2)"/>
    <column name="created_at" type="DATETIME"
defaultValueComputed="CURRENT_TIMESTAMP"/>
  </createTable>
  <addForeignKeyConstraint baseTable="timelog" baseColumn="task_id"
                           referencedTable="task" referencedColumn="id"/>
</changeSet>
```

Payroll Service Schema:

```
<changeSet id="30">
  <createTable tableName="salary_structure">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="tenant_id" type="VARCHAR(36)"/>
    <column name="employee_id" type="VARCHAR(36)"/>
    <column name="basic_pay" type="BIGINT"/>
    <column name="hra" type="BIGINT"/>
    <column name="other_allowances" type="BIGINT"/>
    <column name="created_at" type="DATETIME"
defaultValueComputed="CURRENT_TIMESTAMP"/>
  </createTable>
  <addForeignKeyConstraint baseTable="salary_structure" baseColumn="tenant_id"
                           referencedTable="tenant" referencedColumn="id"/>
</changeSet>
<changeSet id="31">
  <createTable tableName="payroll_run">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="tenant_id" type="VARCHAR(36)"/>
    <column name="period_start" type="DATE"/>
    <column name="period_end" type="DATE"/>
    <column name="executed_by" type="VARCHAR(36)"/>
    <column name="executed_at" type="DATETIME"
defaultValueComputed="CURRENT_TIMESTAMP"/>
  </createTable>
  <addForeignKeyConstraint baseTable="payroll_run" baseColumn="tenant_id"
                           referencedTable="tenant" referencedColumn="id"/>
</changeSet>
<changeSet id="32">
  <createTable tableName="payslip">
    <column name="id" type="VARCHAR(36)" primaryKey="true"/>
    <column name="payroll_run_id" type="VARCHAR(36)"/>
    <column name="employee_id" type="VARCHAR(36)"/>
    <column name="gross_pay" type="BIGINT"/>
    <column name="net_pay" type="BIGINT"/>
```

```
    <column name="created_at" type="DATETIME"
defaultValueComputed="CURRENT_TIMESTAMP"/>
  </createTable>
  <addForeignKeyConstraint baseTable="payslip" baseColumn="payroll_run_id"
                           referencedTable="payroll_run" referencedColumn="id"/>
</changeSet>
```

Liquibase automatically applies these changes. Rollback scripts should also be provided.

5. Keycloak Integration

We use **Keycloak** for SSO/OAuth2. The **auth-service** (Keycloak Provider) will interact with Keycloak's REST API (via the `com.upcraft.keycloak` library) to manage realms, users, and roles. Each service validates JWT tokens on incoming requests (as `oauth2ResourceServer`). For example, add dependency `spring-boot-starter-oauth2-resource-server` and configure issuer URI to Keycloak. Use Keycloak's admin REST API (token grants) to create users/clients programmatically.

Baeldung notes Keycloak as a full OIDC server with admin APIs [1]. We will define a `KeycloakConfig` class in each service or shared library with realm info. The `keycloak-provider` jar will encapsulate common functions (e.g. `createUser`, `assignRole` using Keycloak AdminClient). In code, secure endpoints with `@PreAuthorize("hasRole('ROLE_MANAGER')")` etc, mapping to Keycloak roles.

Each microservice should include Swagger OAuth2 security scheme so APIs are protected in docs.

6. Common DTOs and Swagger

A shared `common-dto` project includes classes used across services (annotated with Lombok or manual getters/setters):

- `TenantDTO`, `UserDTO`, `EmployeeDTO`, `TaskDTO`, `SubtaskDTO`, `TimeLogDTO`, `PayslipDTO`, etc.
- All services include `@ApiResponse` and `@Schema` annotations for Swagger models.

Swagger documentation will be auto-generated (Springdoc) from controllers. Example annotations:

```
@Operation(summary="Create a new Task")
@ApiResponse(responseCode="201", description="Task created")
@PostMapping("/tasks")
public TaskDTO createTask(@RequestBody TaskDTO dto) { ... }
```

This ensures a consolidated OpenAPI spec (via gateway or separate per service).

7. Implementation Plan (AI Prompt)

To generate the code via AI, use the following prompt outline:

AI Prompt (Code Generation):
"Generate a multi-service Spring Boot backend with MySQL and Liquibase. Create the following services and libraries:

1. **keycloak-provider** – A library with Keycloak admin integration (`com.upcraft.keycloak`). Includes utilities to create realms, users, and roles using Keycloak Admin API.
2. **user-service** – Microservice with endpoints for tenant and user management (`/api/users`). Secured via Keycloak. Uses `common-dto.UserDTO`. Includes Liquibase schema for `tenant` and `user`.
3. **employee-service** – Manages Employee entities (`/api/employees`) with CRUD, attendance (`/api/employees/{id}/punch`). Liquibase for `employee` and related tables.
4. **task-service** – Handles tasks (`/api/tasks`) and subtasks, time logs, approvals. Endpoints: create task, list tasks, add time log, approve task. Use `common-dto.TaskDTO`, `TimeLogDTO`. Liquibase for `task`, `subtask`, `timelog`, `attachment`.
5. **payroll-service** – Manages payroll runs (`/api/payroll`) and payslips. Endpoints: run payroll, get payslips. Uses `PayslipDTO`. Liquibase for `salary_structure`, `payroll_run`, `payslip`.
6. **notification-service** – Sends emails/SMS. Exposes `/api/notifications/email` etc. (For demo, stub implementation).

All services should include Swagger (OpenAPI) documentation, secure endpoints with Spring Security and Keycloak JWT. Use Spring Data JPA for MySQL access. Use Liquibase changelogs (provide changesets as XML) to create schemas as above. Configure application.yml for multi-tenant (DB with tenant_id columns).

Implement common error handling (error JSON responses). Provide example JSON requests/responses in controllers.

Also produce:
- `common-dto` library JAR with shared DTOs.
- `gateway-service` (optional) to route requests with Keycloak authentication.
- Dockerfiles and Kubernetes manifests for each service.
- Application pipelines (CI/CD YAML).
- SQL seed data for one sample tenant, user, employee, task.
- A consolidated OpenAPI spec (e.g. via Swagger UI) and a Postman collection JSON of all APIs.

Follow the architecture and API spec as documented above."

Include this prompt in the README so that an AI tool can generate the project code.

---

[1] A Quick Guide to Using Keycloak with Spring Boot | Baeldung
https://www.baeldung.com/spring-boot-keycloak